VIDHUN VIJAYAKUMAR SUJA

Edinburgh, UK | vsvidhun06@gmail.com | +44 7553 039813
linkedin.com/in/vidhun-suja | github.com/vsvidhun06-blip | vsvidhun06-blip.github.io | leetcode.com/u/VidhuViny

SUMMARY

MSc Software Engineering graduate (Heriot-Watt, June 2026) specialising in LLM serving infrastructure and concurrent systems. Built Mini-vLLM — a from-scratch inference engine implementing continuous batching, paged KV cache, prefix caching, GPU acceleration, and speculative decoding on TinyLlama-1.1B, validated byte-identical to HuggingFace (3.27× scheduler throughput, 2.0× prefill speedup, 63% prefix cache hit rate). MSc dissertation on weak memory models and interactive concurrent execution visualisation under Prof. Marko Doko.

TECHNICAL SKILLS

LLM Infrastructure: PyTorch, CUDA, Continuous Batching, Paged Attention, Prefix Caching, Speculative Decoding, KV Cache Management, SSE Streaming, HuggingFace, vLLM-pattern serving

Languages: Python, Java, TypeScript, JavaScript, SQL, C/C++

Backend & Distributed Systems: FastAPI, Spring Boot, RESTful APIs, WebSocket, Microservices, Kafka, Circuit Breakers (Resilience4j), Service Discovery (Eureka), Rate Limiting, Thread Pool Tuning

Infrastructure & Observability: Docker, Docker Compose, GitHub Actions CI/CD, Prometheus, Redis, PostgreSQL, k6 Load Testing

Concurrency & Formal Methods: Weak Memory Models (C11/C++11, ARM, POWER), Happens-Before Reasoning, Race Condition Analysis, Graph Algorithms, Async I/O, Connection Pooling

EXPERIENCE

Full-Stack Software Engineer Intern *Kompetenzen Technologies, India | Dec 2023 – Aug 2024*

- Engineered multiple production modules across a distributed Java/React platform, driving 30% reduction in P95 API latency and 15% uplift in user engagement through performance profiling, thread pool tuning, and executor optimisation.
- Designed high-throughput RESTful APIs serving 500+ concurrent sessions; increased data processing throughput 25% via connection-level optimisations and async patterns; load-tested with k6 to validate performance under sustained traffic.
- Reduced concurrent session failures 95% by redesigning Spring Boot session management with idempotent request handling and exponential backoff, restoring 99%+ uptime.

PROJECTS

Mini-vLLM — From-scratch LLM Inference Engine *Apr 2026 – May 2026*

Stack: *Python, PyTorch, CUDA, FastAPI, WebSocket, Prometheus, D3.js* **| GitHub:** github.com/vsvidhun06-blip/mini-vllm

- Built vLLM-pattern continuous batching scheduler with paged KV cache (block_size=16, layer-major K/V split, admission control) for TinyLlama-1.1B; measured 3.27× CPU scheduler throughput on 4 concurrent requests and 1.95× batched-vs-solo speedup on RTX 4060 GPU.
- Implemented reference-counted prefix cache with chained position-aware block hashing; measured 2.0× average prefill speedup and 63.2% cache hit rate on shared-prefix workloads (129-token system prompt, 4 concurrent requests).
- Built speculative decoding with configurable self-speculation; characterised draft acceptance across (K, exit_layer) configurations and identified K=2/exit_layer=18 as best-performing config (37.5% acceptance); designed pluggable draft interface for trained-head extensions (EAGLE/Medusa).
- Verified byte-identical correctness against HuggingFace reference via parity tests at every engine layer (forward, KV cache, generation, batching, prefix sharing); instrumented Prometheus /metrics (TTFT/TPOT/p50-p95-p99 histograms) and live D3 WebSocket visualiser.

Interactive Exploration of Concurrent Programs' Executions (MSc Dissertation) *Sep 2025 – Apr 2026*

Stack: *Java 21, JavaFX, Graph Algorithms, Formal Methods, C11/C++11 Memory Models*

Supervised by Prof. Marko Doko, Heriot-Watt University

- Built Java 21/JavaFX desktop tool for interactive step-through of concurrent program executions; renders memory operations as live graph nodes with happens-before and synchronises-with edges updating in real time.
- Closes a gap left by herd7 and CDSChecker — guides step-by-step construction of each execution rather than enumerating all outcomes, making it easier to reason about why specific outcomes are allowed or forbidden on ARM/POWER weak memory hardware.
- Validated correctness across 80/80 model-test combinations covering SC, TSO, PSO, RA, and WEAKEST memory models; structured user study (n=9) measured time-to-understanding for advanced concurrency concepts.

Cloud-Native E-Commerce Microservices Platform *Dec 2025 – Feb 2026*

Stack: *Java, Spring Boot, Kafka, Docker Compose, PostgreSQL, Redis, Eureka, Resilience4j, GitHub Actions* **| GitHub:** github.com/vsvidhun06-blip/ecommerce-microservices

- Designed distributed e-commerce backend across independently deployable Spring Boot microservices orchestrated with Docker Compose; integrated Kafka event streaming, Eureka service discovery, Resilience4j circuit breakers, and tuned thread pools for consistent throughput under load.
- Implemented Redis caching layer reducing backend DB load on hot paths; designed collaborative filtering recommendation engine over user-item interaction data; automated build, test, and image publishing pipeline via GitHub Actions.

ACHIEVEMENTS

- Competitive Programming: 153 LeetCode problems solved (33 Easy, 93 Medium, 27 Hard) across arrays, dynamic programming, backtracking, graphs, and binary search.

EDUCATION

MSc Software Engineering — Heriot-Watt University, Edinburgh, UK *Expected June 2026*

BTech Computer Science — College of Engineering Muttathara, Kerala, India *July 2023*